LEMu: Live Electronic Music Generator

Rene Wooller, Nick Coleman and Andrew R. Brown

Queensland University of Technology
Victoria Park Rd.
Kelvin Grove, 4059. Australia

r.wooller@student.qut.edu.au yellowrobot@hotmail.com
a.brown@qut.edu.au

This paper examines the software instrument "LEMu", which is being developed by the authors at the Queensland University of Technology. In this paper we will answer the questions: "What is LEMu?", "What is it designed to do?", "How does it work?", "What music does it produce?" and "What are some future developments of LEMu?". Many types of intelligent instruments are emerging from both the academic and commercial worlds. For example there is the interactive algorithmic composition *AutoGam*, developed by Thierry Bachman and the Korg *KARMA* keyboard, developed by Stephen Kay. Presently the commercial dance scene lacks (to some extent) real time performance, creation and improvisation. LEMu encourages DJ's and electronic musicians to explore these concepts within the context of the dance music scene.

What is LEMu?

LEMu is a software instrument that works with MIDI events. It has the ability to create musical phrases and manipulate them in real time. It is predisposed to creating music for many underground and commercial genres of electronic music such as techno, house, trance, breakbeat and electro.

As LEMu works within the MIDI realm, a lot of the flexibility and musicality of the instrument depends on what type of synthesizer is creating the sounds. In our own MIDI hardware setup, we use the Alesis DM5 drum machine, the Roland JP8080 analogue modelling synthesizer and the Kurzweil K2000, with sounds that we have programmed in ourselves. This setup along with our musical understanding defines our own unique LEMu sound.

LEMu was programmed in the jMusic environment that is an extension of the Java programming language. jMusic was developed by Andrew Brown and Andrew Sorensen and was introduced to ACMA at Interfaces ACMC 2000 (Sorensen & Brown 2000).

What is it designed to do?

LEMu is a fusion of two major electronic music concepts – the academic and the commercial. As an academic concept it was created to more fully explore various interesting ideas concerning computer music. These are: realtime jMusic, aleatoric composition and mathematic manipulation of sequences. In respect to the commercial possibilities of the software, LEMu offers techno/dance music artists and producers, a new live performance and music creation system. This system grants the user many spontaneous performance abilities, some of which are similar to standard DJ techniques (layering of tracks, cross fading, slicing). Important concepts for this goal were: interaction, useable interface and design, storage and playback of tracks/patterns. In time, as LEMu evolves, it may gain support in the club scene where it will exist as an alternative medium for live performance of electronic music.

How does it work?

On opening the program the user is presented with the master interface.

From the master interface, the user can create up to sixteen sequence modules and synchronise the timing of the program (the speed may need adjustment on various different platforms). During playback, the master interface has general controls for the manipulation of the music such as tempo and master transposition. It also allows for the saving and storage of MIDI files (capturing a live performance). There are two types of sequencing modules – rhythm and pitch.

Upon creation the sequence modules are assigned to a MIDI channel. MIDI data from each module is compiled into a beat long score that is played using the QuickTime utility object from jMusic. QuickTime then routes the data to external MIDI hardware.

Pitch module

This module is concerned with the creation and manipulation of musical phrases. Universal module parameters include sound selection, pattern length, time signature, note density, mode, pitch set, phase offset, transpose, rhythmic shift, dynamic and mute state. These are controlled by the user and have values that are independent of the other modules. Phase offset offsets the start time of the phrase by semiquaver increments.

Rhythm module

The rhythm module starts with a preset pattern, which is looped. User can control sound selection and drum rhythms. We have defined eight parts to a drum module: kick, snare, high hats open, high hats closed, clap, cymbal, aux1, aux2. The preset pattern has a relevant sequence for each of these parts, the style menu option dictating the patterns necessary for each part. Each of these parts can be assigned to a MIDI note number so really the kick part could in fact be playing any sound if the user wishes to set it up in such a way.
Each part has a rhythmic shift slider and a mute button. The shift slider creates a polyrhythmic version of the part that is affected. It defaults to a value of four, signifying four beats to a bar, and can move from 1 to 8.

Capture

The capture function in the master interface is used to merge all the beat long scores into a single score that is then converted to a standard MIDI file.

What music does it produce?

The music created with LEMu has its own trademark. By this we mean that there are a few compositional ideas that are generally consistent from one song to the next. These are: layering, repetition, polyrhythms and the preset beats that are derived from the electronic dance music genre. These are the functions that LEMu performs most easily, thus defining the style of the instrument.

Future development

Most of the features that we have developed in LEMu will be refined and many new features are yet to be added. As there is currently no physical controller input into the software, effective manipulation is limited by mouse and keyboard operation. We have attempted to make the software interface more usable, which is necessary if LEMu is to be run independently. Some of the ideas we are yet to implement are:

interaction between parts, a more intelligent approach to the creation of musical phrases, a variety of algorithms to select from for the shift sliders (of which the current "polyrhythmic algorithm" will be one) and a chord/arpeggiator module.

In light of the release of Mac OSX we will be looking to develop LEMu using Java 1.3. The most obvious reason for us doing this would be that 1.3 includes the JavaSound libraries which can be used instead of QuickTime for MIDI communication and to add audio features. LEMu has been programmed in such a way that outputting to realtime jMusic Audio (available with 1.3 of Java and 1.1 of jMusic) is trivial. However the extension of LEMu into the jMusic Audio realm will be put on hold until most, if not all of our features have been developed further.

As it is a live performance tool, live manipulation techniques also have to be coherent and usable, with respect to all controller sources. The program has also lacked sufficient testing on different platforms, something that will be done by the end of this year.

Summary

Although happy with many of the concepts discussed in this paper, the next six months will see most of these ideas either refined or developed. To create a coherent and user-friendly program, with a commercial prospect in mind, we need to filter complex ideas into simplified user controlled parameters and develop a better understanding of programming methods and concepts. As musicians, we see that the development of LEMu is creating some new interpretations of electronic music allowing us to experiment with and realise our musical ideas

Bibliography

Brown, A. R. and Sorenson, A. (2000) "Introducing jMusic" in Brown A. R. and Wilding, R. (ed) *Interfaces: Proceedings of the Australasian Computer Music Conference 2000*. Brisbane: ACMA

Cooper, G. and Meyer, L. (1960) *The Rhythmic Structure of Music*. Chicago: The University of Chicago Press

Dean, R. (1989) *Creative Improvisation*. Philadelphia: Open University Press

Loy, G. (1988) "Composing with Computers - a Survey of Some Compositional

Formalisms and Music Programming Languages" in Mathews, M.V. and Pierce, J.R. (eds.) *Current Directions in Computer Music Research*. London: The MIT Press, pp.294-297

Pressing, J. (1992) *Synthesizer Performance and Real-Time Techniques*. Oxford: Oxford University Press

Pope, S.T. (1993) "Real – Time Performance via User Interfaces to Musical Structures", CREATE, URL:

http://www.create.ucsb.edu/~stp/PisaPaper/Pisa.html (15, 06, 01)

Roads, C. (1996) *The Computer Music Tutorials*. London: The MIT Press

Sachs, C. (1953) *Rhythm and Tempo: A Study in Music History.* New York: Vail-Ballou Press

Yu, C. (1996) "Computer Generated Music Composition", MIT

URL: http://www.oz.net/~cyu/thesis (15, 06, 01).

Brown, A. R. and Sorenson, A. (2001) "jMusic: Music Composition in Java", QUT
URL:http://www.academy.qut.edu.au/music/newmedia/jMusic/